Discord Authenticator – Bericht

Joel Damm (jd106), Joshua Jung (jj050)

Inhalt

1. Projektziel und Problemstellung **2**
2. Anforderungen **3**
2.1 Muss-Ziele (funktionale Kernanforderungen) 3
2.2 Kann-Ziele (Qualitäts- und Architekturziele) 4
3. Systemarchitektur **5**
3.1 Gesamtarchitektur 5
3.2 Reverse Proxy (Caddy) 6
4. OAuth2-Authentifizierungsprozess **7**
Ablauf: 7
5. Schlüsselverwaltung und Verschlüsselung **9**
5.1 Token-Verschlüsselung 9
5.2 Schlüsselverwaltung 10
6. Erfüllung der Schutzziele (CIA-Modell) **12**
6.1 Vertraulichkeit 12
6.2 Integrität 13
6.3 Verfügbarkeit 13
7. Verwendete Technologien und Libraries **14**
Backend 14
Infrastruktur 14
8. Herausforderungen und systemische Einordnung **15**
9. Reflexion und Fazit **18**

1. Projektziel und Problemstellung

Ziel dieses Projekts war die Entwicklung eines eigenständigen Authentifizierungsdienstes, der Nutzern ermöglicht, sich mittels ihres Discord-Accounts bei einer Webanwendung anzumelden.

Der Dienst implementiert den OAuth-2.0 Authorization Code Flow mit PKCE und orientiert sich an industriellen Sicherheitsstandards. Ziel war nicht nur eine funktionierende Anmeldung, sondern eine sicherheitsbewusste, skalierbare und erweiterbare Architektur.

Das System wurde als Full-Stack-Lösung konzipiert und umfasst:

- Backend (Rust, Axum)
- Frontend (SvelteKit)
- PostgreSQL (persistente Nutzerdaten)
- Redis (Session-Management)
- Reverse Proxy (Caddy mit TLS)
- Containerisierung mit Docker

Die Datei “README.md” im Git-Repository enthält die Anleitung zur Installation.

2. OAuth2-Authentifizierungsprozess

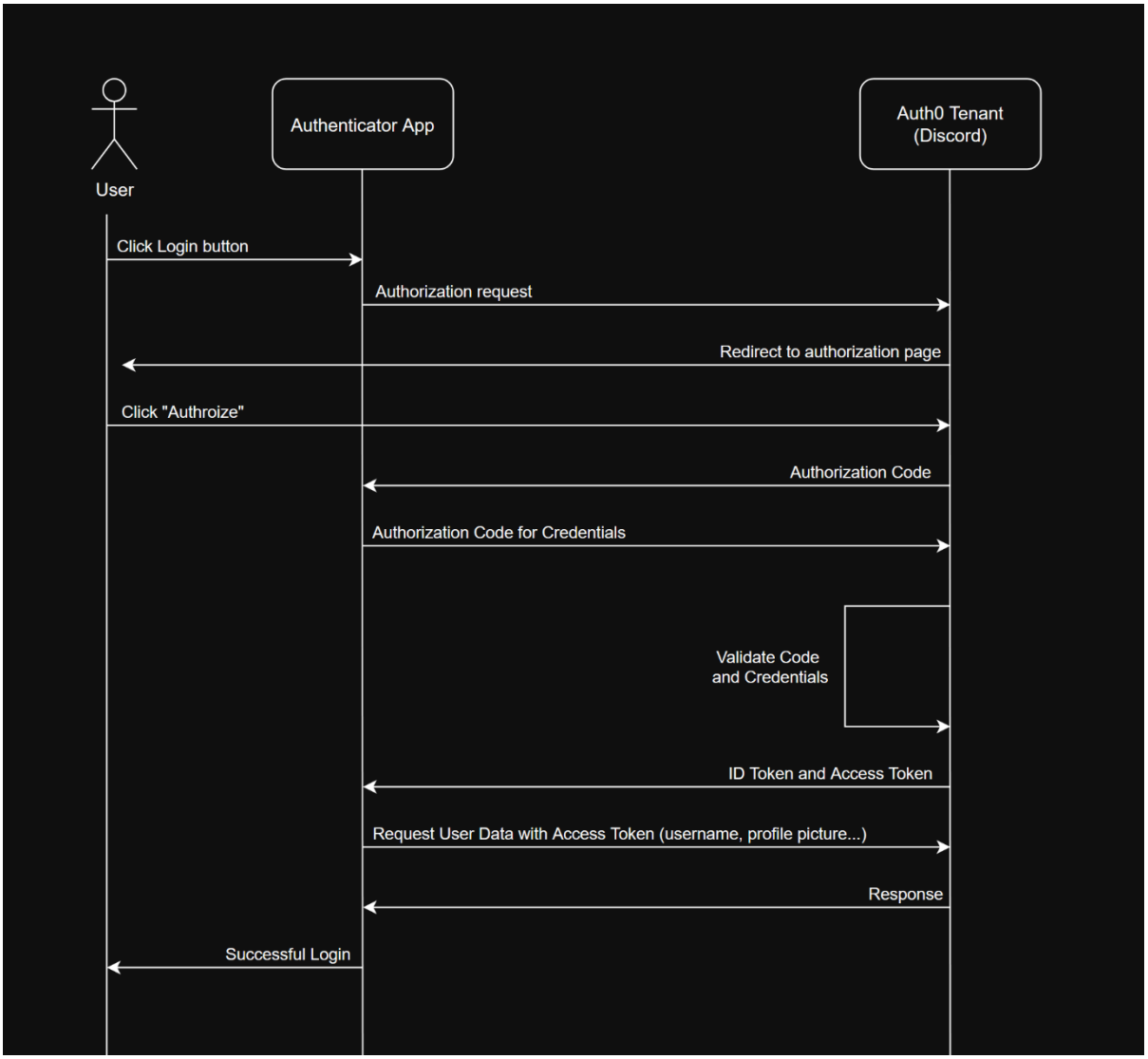
Der Authentifizierungsprozess basiert auf dem Authorization Code Flow mit PKCE.
Basierend auf: https://oauth.net/2/

Ablauf:

1. Erzeugung von PKCE-Challenge, CSRF-Token und Session-ID
2. Speicherung der temporären OAuth-Session in Redis
3. Weiterleitung zu Discord
4. Callback mit Authorization Code
5. Validierung von:
    - CSRF-Token
    - PKCE-Verifier
6. Austausch des Codes gegen Access- und Refresh-Token
7. Abruf von Benutzerdaten
8. Persistierung in PostgreSQL
9. Erzeugung einer serverseitigen Session in Redis
10. Übergabe eines HttpOnly-Session-Cookies an das Frontend

Es werden zu keinem Zeitpunkt Benutzerpasswörter verarbeitet oder gespeichert.

Abb. 1 OAuth2 flow, wie beschrieben auf: https://aaronparecki.com/oauth-2-simplified/



3. Anforderungen

3.1 Muss-Ziele (funktionale Kernanforderungen)

Diese Anforderungen waren zwingend notwendig:

- OAuth2 Authorization Code Flow
- PKCE-Absicherung
- CSRF-Schutz
- Sichere Session-Verwaltung
- HttpOnly- und Secure-Cookies
- TLS-gesicherte Kommunikation
- Persistente Speicherung von Nutzern
- Trennung von Code und Secrets

Ohne diese Anforderungen wäre das System nicht sicher betreibbar.

3.2 Kann-Ziele (Qualitäts- und Architekturziele)

Diese Ziele erhöhen Robustheit und Professionalität:

- AES-256-GCM-Verschlüsselung von Refresh-Tokens
- TTL-basierte Session-Verlängerung
- Compile-Time Query Validation mit SQLx
- Containerisierte Infrastruktur
- Logging via tracing
- Admin-Tools (pgAdmin, RedisInsight)
- Monorepo-Architektur

4. Systemarchitektur

4.1 Gesamtarchitektur

Das System ist in klar getrennte Komponenten aufgeteilt:

1. Reverse Proxy (Caddy)
2. Auth-Service (Rust)
3. PostgreSQL-Datenbank
4. Redis Session Store
5. Frontend (SvelteKit)

Die Services kommunizieren innerhalb eines isolierten Docker-Netzwerks.

Diese Trennung ermöglicht:

- Skalierbarkeit
- Wartbarkeit
- Fehlerisolation
- klare Sicherheitsgrenzen

4.2 Reverse Proxy (Caddy)

Der Reverse Proxy übernimmt:

- TLS-Terminierung
- HTTPS-Weiterleitung
- Zertifikatsmanagement (ACME)
- Weiterleitung interner Services

Sicherheitswirkung:

- Schutz vor Man-in-the-Middle-Angriffen
- Erzwingen verschlüsselter Verbindungen
- zentrale Sicherheitsgrenze (Security Boundary)

5. Schlüsselverwaltung und Verschlüsselung

5.1 Token-Verschlüsselung

Refresh-Tokens werden mit AES-256-GCM verschlüsselt gespeichert.

Verwendete Library: aes-gcm

Eigenschaften von AES-256-GCM:

- 256-Bit symmetrischer Schlüssel
- Authenticated Encryption (Integrität + Vertraulichkeit)
- 96-Bit Nonce
- Manipulationserkennung durch Authentication Tag

Bei Speicherung werden erzeugt:

- Ciphertext
- zufällige Nonce

Ohne korrekten Schlüssel ist eine Entschlüsselung kryptographisch nicht praktikabel.

5.2 Schlüsselverwaltung

Der Verschlüsselungsschlüssel:

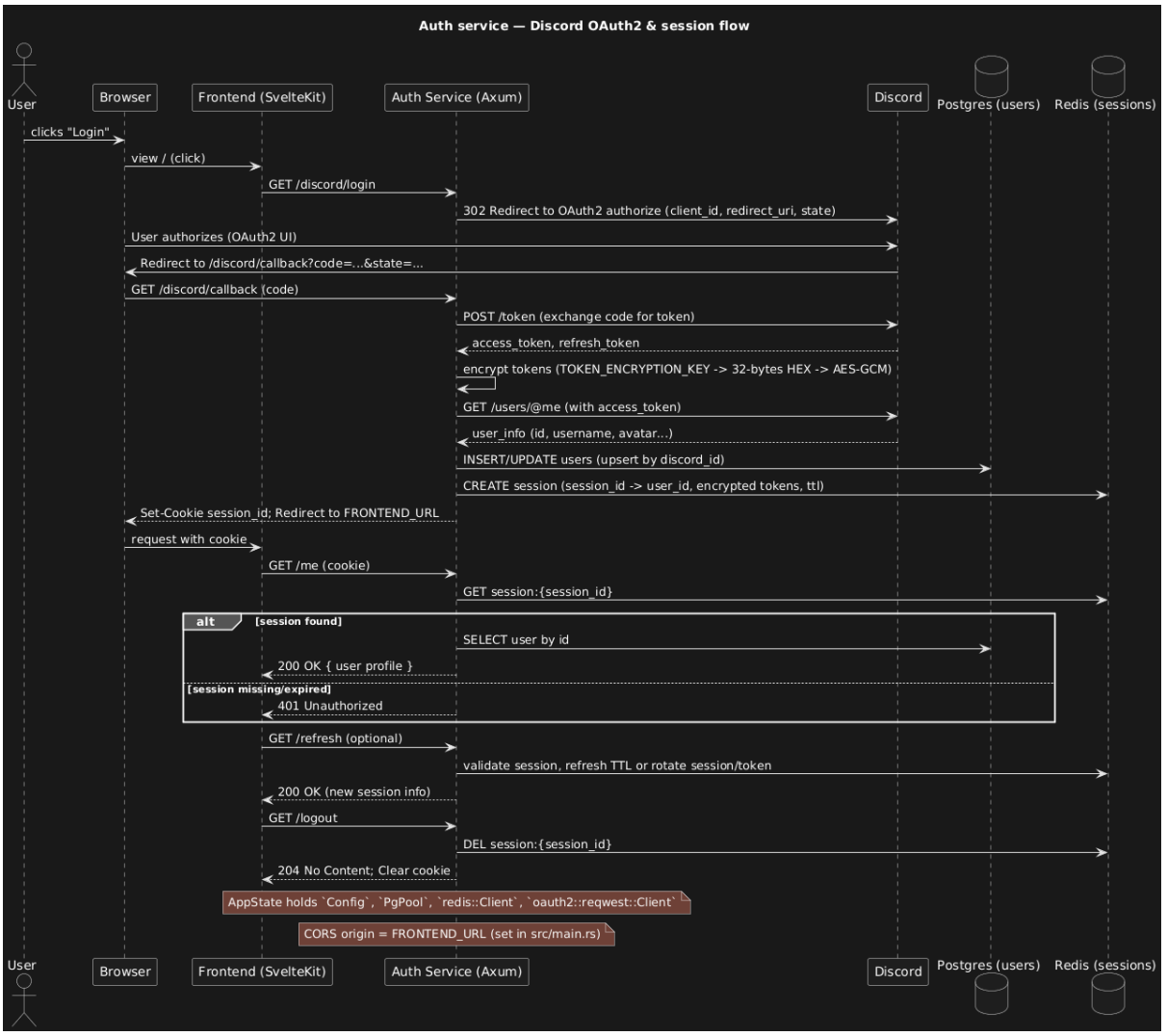
- wird als 32-Byte HEX-Wert gespeichert
- liegt in einer Environment-Variable (TOKEN_ENCRYPTION_KEY)
- wird über Docker Compose injiziert
- ist nicht im Repository enthalten

Damit wird erreicht:

- Trennung von Code und Secrets
- Austauschbarkeit des Schlüssels
- keine Hardcoded-Secrets

Für produktive Systeme wären zusätzlich Secret-Manager oder Docker-Secrets empfehlenswert.

Abb.2: Sequenzdiagramm des Auth-Service, dargestellt sind Anmeldung und Abmeldung.



6. Erfüllung der Schutzziele (CIA-Modell)

6.1 Vertraulichkeit

Erreicht durch:

- TLS-Verschlüsselung (Caddy)
- Secure-Cookies
- HttpOnly-Cookies
- AES-256-GCM für Tokens
- Environment-basierte Secret-Verwaltung

Schützt vor:

- Abhören von Daten
- Token-Diebstahl via XSS
- Klartextspeicherung sensibler Daten

6.2 Integrität

Erreicht durch:

- PKCE-Mechanismus
- CSRF-State-Validierung
- GCM-Authentifizierungstag
- Typensicherheit durch Rust
- Compile-Time SQL-Prüfung (SQLx)

Schützt vor:

- Code-Injection im OAuth-Flow
- manipulierten Tokens
- SQL-Injection
- unvalidierten Datenbankzugriffen

6.3 Verfügbarkeit

Erreicht durch:

- Containerisierte Services
- Redis für schnelle Session-Verwaltung
- Trennung von Persistenz und Session Layer
- isoliertes Docker-Netzwerk

Erhöht:

- Skalierbarkeit
- Wartbarkeit
- Fehlerisolierung

7. Verwendete Technologien und Libraries

Backend

- axum – Web Framework
- tokio – Async Runtime
- oauth2 – OAuth2-Implementierung
- sqlx – Compile-Time Query Validation
- redis – Async Session Store
- aes-gcm – Kryptographische Verschlüsselung
- serde / serde_json – Serialisierung
- uuid – Session-ID Generierung
- rand – kryptographische Nonce-Erzeugung
- tower-http – CORS Middleware
- tracing – Logging

Infrastruktur

- PostgreSQL
- Redis
- Docker & Docker Compose
- Caddy Reverse Proxy
- SvelteKit (Frontend)

8. Herausforderungen und systemische Einordnung

Während der Umsetzung zeigte sich, dass die Hauptschwierigkeit nicht in einzelnen Technologien lag, sondern in der Integration mehrerer sicherheitskritischer Komponenten innerhalb einer verteilten Architektur.

Die folgenden Herausforderungen lassen sich sowohl technisch als auch aus Sicht von Software-Engineering-Prinzipien analysieren:

1. Komplexität des OAuth2 Authorization Code Flow (Security by Design)

Die Implementierung des OAuth2 Authorization Code Flow mit PKCE erforderte ein präzises Verständnis der zugrundeliegenden Sicherheitsmechanismen.

Kritische Komponenten:

- PKCE Challenge/Verifier
- CSRF-State-Validierung
- Authorization Code Exchange
- Token-Handling
- Redirect-URI-Konsistenz

Bereits kleine Inkonsistenzen führten zu nicht offensichtlichen Fehlerbildern. Dies verdeutlichte das Prinzip:

Security by Design – Sicherheitsmechanismen dürfen nicht nachträglich ergänzt, sondern müssen von Beginn an korrekt integriert werden.

Zudem zeigte sich hier die Notwendigkeit von:

- Klarer Prozessmodellierung
- Sequenzdiagrammen
- formaler Ablaufvalidierung

2. Cookie-Handling hinter Reverse Proxy (Defense in Depth)

Die Konfiguration von Cookies in Kombination mit einem Reverse Proxy stellte eine klassische Mehrschichten-Herausforderung dar.

Beteiligte Ebenen:

- Browser (SameSite, Secure, HttpOnly)
- Backend (Cookie-Setzung)
- Reverse Proxy (TLS-Terminierung)
- CORS-Policy
- Frontend (Fetch mit Credentials)

Hier wurde deutlich, dass Web-Sicherheit immer mehrschichtig ist. Dieses Problem lässt sich mit dem Prinzip “Defense in Depth” (Mehrschichtige Sicherheitsarchitektur) erklären.

Jede Schicht (TLS, Cookies, CSRF, PKCE) übernimmt einen Teil der Sicherheitsverantwortung.

Eine fehlerhafte Konfiguration auf einer Ebene kann das Gesamtsystem kompromittieren.

3. Datenbankmigrationen und Compile-Time Validation (Correctness over Convenience)

SQLx bietet Compile-Time Query Validation.

Das erhöht die Typ- und Schema-Sicherheit erheblich, erzwingt jedoch strikte Migrationsdisziplin.

Fehler wie: „migration was previously applied but has been modified“ zeigen, dass:

- Schemaänderungen deterministisch versioniert werden müssen
- Datenbankzustände reproduzierbar sein müssen

Dies verweist auf das Prinzip “Reproducibility & Deterministic Builds”. Moderne Softwareentwicklung erfordert reproduzierbare Zustände – sowohl für Code als auch für Datenbankschemata.

4. Containerisierung und Build-Zeiten (Trade-off zwischen Portabilität und Iteration)

Docker erhöhte:

- Isolation
- Portabilität
- Deployment-Konsistenz

Gleichzeitig verlängerten sich:

- Build-Zeiten
- Iterationszyklen
- Debugging-Prozesse

Hier zeigte sich ein klassischer Engineering-Trade-off: Optimierung für Deployment vs. Optimierung für Entwicklungsiteration. Containerisierung verbessert Systemkonsistenz, verlangsamt jedoch lokale Entwicklungszyklen, wenn sie früh eingeführt wird.

5. Systemische Komplexität

Die größte Lernerfahrung war das Verständnis von Emergenter Komplexität. Jede einzelne Komponente (OAuth2, Redis, PostgreSQL, Reverse Proxy, Docker) ist für sich überschaubar.

Die Kombination dieser Systeme erzeugt jedoch eine deutlich höhere Gesamtsystemkomplexität. Diese Erfahrung verdeutlicht zentrale Prinzipien moderner Softwarearchitektur:

- Separation of Concerns
- Security by Design
- Defense in Depth
- Reproducibility
- Explicit Configuration

9. Reflexion und Fazit

Die zentralen Herausforderungen lagen weniger in einzelnen Implementierungsdetails als in der sicheren Integration mehrerer Systeme, der Beherrschung verteilter Zustände sowie der Abstimmung zwischen Infrastruktur- und Anwendungsebene.

Durch iterative Analyse, strukturierte Architekturplanung und den Einsatz sicherer Libraries konnte ein stabiles und sicherheitsorientiertes Gesamtsystem realisiert werden. Das Projekt vereint IT-Sicherheit, moderne Webarchitektur, Containerisierung, Datenbankdesign und OAuth2-Standards.

Zentrale Erkenntnisse

- Security muss von Beginn an architektonisch berücksichtigt werden.
- Compile-Time-Checks reduzieren Runtime-Fehler signifikant.
- Secrets müssen strikt vom Code getrennt werden.
- Industriestandards wie PKCE und AES-GCM sind essenziell für sichere Systeme.

Das System erfüllt die grundlegenden Schutzziele der Informationssicherheit (Vertraulichkeit, Integrität, Verfügbarkeit), orientiert sich an Best Practices und bietet eine skalierbare Grundlage für weitere OAuth-Provider oder Frontends.

Verbesserungspotenzial für Folgeprojekte

1. Frühzeitige Architekturmodellierung

Klare Systemgrenzen, dokumentierte Datenflüsse und früh definierte Schutzziele („Architecture before Implementation“).

2. Saubere Konfigurationstrennung

Konsequente Trennung von Dev/Staging/Production, Einsatz von Secret-Management („Configuration as Code“).

3. Strategischere Containerisierung

Kernlogik zunächst lokal stabilisieren, Containerisierung später optimieren („Optimize for Iteration First“).

4. Früh etablierte Teststrategie

Integrationstests für Auth-Flow, Session-Logik und Migrationen („Testability as Architectural Requirement“).

5. Ausbau von Observability und Security-Review

Strukturierte Logs, Monitoring sowie formales Threat Modeling vor Deployment.

Schlussfolgerung

Die Komplexität moderner Web-Authentifizierung entsteht primär durch Mehrschichtigkeit, Sicherheitsanforderungen und Infrastrukturintegration – nicht durch einzelne Technologien.

Ein zukünftiges Projekt würde daher noch stärker auf formale Architekturplanung, reproduzierbare Infrastruktur, systematische Tests und dokumentierte Sicherheitsanalysen setzen.